

STEGANOGRAPHY

DETECTION SYSTEM

Universal Forensic Analyzer

Images · Documents · Audio · Video

Full Project Technical Explanation

Prepared: June 28, 2026

What This Project Does

This system detects steganography — the art of hiding secret data inside ordinary-looking files. It can analyse images, documents, audio files, and video files, and tell you whether hidden data is present, how suspicious the file is, and what technique was used to hide the data.

It produces a full PDF forensic report and has a clean graphical interface that anyone can use without technical knowledge.

■ Images	■ Documents	■ Audio	■ Video
Images	Documents	Audio	Video
PNG · JPG	PDF · DOCX	WAV · MP3	MP4 · AVI
BMP · TIFF	PPTX · XLSX	FLAC · OGG	MKV · MOV
WEBP · GIF	TXT · CSV · MD	M4A · AAC	WMV · FLV

	HTML · XML · RTF	AIFF	WEBM
--	------------------	------	------

1. What is Steganography?

Steganography comes from the Greek words *steganos* (hidden) and *graphia* (writing). It is the practice of concealing secret information inside an ordinary, non-secret file or message so that no one apart from the sender and receiver knows the communication is happening at all.

Technique	What It Does
Steganography	Hides the EXISTENCE of data. File looks completely normal.
Encryption	Hides the CONTENT of data. Everyone knows it is encrypted.
Watermarking	Embeds an owner tag invisibly. Similar technique, different purpose.

Real-World Examples

- A photo of a cat that secretly contains a text message hidden in its pixels
- A Word document with white text on white background containing passwords
- An audio file with secret data embedded in sound sample bits
- A video file with extra data appended after the video ends
- A PDF with invisible text layer containing hidden instructions

Who Uses It?

Cybercriminals	Exfiltrate stolen data past security systems without detection
Malware authors	Hide malicious payloads inside innocent-looking image files
Journalists	Securely transmit sensitive information in authoritarian regimes
Military / Intel	Covert communication channels that appear as normal traffic
Digital forensics	Detect hidden data in evidence during criminal investigations
Copyright protection	Embed invisible watermarks to track illegal distribution

2. Project Structure and Architecture

The project follows a clean three-layer architecture: the Analysis Layer detects steganography, the GUI Layer displays results to the user, and the Utils Layer handles visualisation and reporting.

File	Purpose
main.py	Entry point — launches the application
analysis/detector.py	Image steganography: LSB, RS, Chi-square, Histogram, DCT
analysis/doc_detector.py	Document/Audio/Video steganography detection
gui/app.py	Graphical User Interface (Tkinter)
utils/visualizer.py	Creates charts, LSB plane images, histograms
utils/report.py	Generates professional PDF forensic reports
requirements.txt	List of all Python libraries the project needs

Technology Stack

Library	Purpose
Python 3.9+	Core programming language
Tkinter	GUI framework — built into Python, no extra install
Pillow (PIL)	Image loading, conversion, thumbnail creation
NumPy	Fast numerical array operations on pixel/sample data
SciPy	Statistical tests (Chi-square p-value calculation)
scikit-image	Image quality metrics and RS analysis functions
OpenCV (cv2)	Video frame extraction and image processing
Matplotlib	Chart and graph generation for visualisation
ReportLab	Professional PDF report generation
PyMuPDF (fitz)	PDF internal structure analysis
python-docx	DOCX file hidden text and structure analysis
python-pptx	PPTX hidden slides and off-canvas object detection
openpyxl	XLSX hidden sheets, rows, columns, cell analysis
librosa	Audio loading and signal processing for WAV/MP3/FLAC
soundfile	High-quality audio file reading

3. Image Steganography Detection

Images are made of pixels. Each pixel has three colour channels: Red, Green, and Blue (RGB). Each channel stores a value from 0 to 255, represented in binary as 8 bits. For example, Red=200 = 11001000 in binary. The **last bit (Least Significant Bit)** changes the colour by only 1/255 — completely invisible to the human eye. This is how data is hidden.

Binary Example

Original pixel Red value: 200 = **11001000**

After hiding bit '1': 201 = **11001001**

Colour change: 200 vs 201 → completely invisible to human eye

The last bit (shown in colour) carries the hidden data. 3 colour channels × 8 pixels = 24 bits = 3 characters of hidden text per 8 pixels.

Detection Technique 1 — LSB Analysis (Least Significant Bit)

Natural images have correlated neighbouring pixels — a blue sky gradually changes colour, a face blends smoothly. This means the LSBs of neighbouring pixels are NOT random — they follow the image structure. When data is hidden, the LSBs become completely random, destroying this natural correlation.

Metric measured	Spatial correlation of LSB values across neighbouring pixels
Normal image	Correlation ~0.5-0.8 (neighbouring pixels related)
Stego image	Correlation ~0.0 (LSBs replaced with random data bits)
Output	avg_correlation value + DETECTED / CLEAN verdict

Detection Technique 2 — Chi-Square Test

When LSB embedding hides data, it causes a specific statistical effect: pixel values that come in pairs (e.g. 100 and 101) get their counts equalised. If 1000 pixels had value 100 and 200 pixels had value 101, after embedding they both become ~600. The Chi-square test measures this unnatural equalisation.

Metric measured	Whether paired pixel value frequencies are suspiciously equal
p-value near 1.0	Natural image — pairs are unequal as expected
p-value near 0.0	Stego image — pairs are equalised by embedding
Threshold	$p < 0.05$ flagged as suspicious

Detection Technique 3 — RS Analysis (Regular-Singular)

RS Analysis is one of the most accurate methods. It divides the image into small groups of pixels, then applies a flipping function (+1 or -1 to LSBs) and measures smoothness. Groups are classified as Regular (R), Singular (S), or Unusable. In a clean image, $R > S$. Hidden data causes R and S to converge. The ratio of R to S reveals the estimated payload percentage.

Key output	Estimated % of image pixels used to store hidden data
Clean image	R much greater than S — imbalanced

Stego image	R approximately equals S — balanced by embedding
Advantage	Can estimate payload size even without knowing what was hidden

Detection Technique 4 — Histogram Analysis

A histogram counts how many pixels have each possible value (0 to 255). A normal image has an irregular, unique histogram. LSB embedding creates a characteristic 'comb' pattern where neighbouring values (100/101, 102/103) become identical — because embedding toggles between them.

Detection Technique 5 — DCT Analysis (Discrete Cosine Transform)

JPEG images use DCT to compress data. Steganography tools (like JSteg, OutGuess) hide data in DCT coefficients. The system applies a Fourier Transform to detect unusual patterns in the frequency domain — particularly elevated high-frequency components that indicate data has been injected into the frequency coefficients.

Message Extraction

If steganography is detected, the system actually reads the hidden message. It extracts the LSB from every pixel in the image, groups the bits into bytes, converts bytes to ASCII characters, and checks if the result contains readable text (printable character ratio > 70%). It tries all colour channels (R, G, B) and reports the best result.

4. Document Steganography Detection

Documents hide data differently from images. Instead of pixel manipulation, documents exploit formatting, invisible characters, metadata, and structural features of file formats. DOCX, PPTX, and XLSX are actually ZIP archives containing XML files — this gives many places to hide data.

PDF Detection — 3 Checks

- **Raw Stream Scan:** Searches the raw PDF binary for suspicious keywords like /JavaScript, /EmbeddedFile, /Launch, /OpenAction. These can trigger code or embed hidden files inside the PDF structure.
- **Invisible Text:** Uses PyMuPDF to read every text element on every page. Checks if any text has white colour (RGB 16777215) or font size less than 0.5pt — completely invisible but present in the file.
- **Blank Pages:** Counts pages with less than 5 characters. A high ratio of blank pages is suspicious — data may be hidden on 'empty' pages.

DOCX / ODT Detection — 5 Checks

- **Hidden Text (Vanish):** Word documents have a 'vanish' property on text runs (w:vanish XML tag). Text with this property is hidden — not visible when printed or viewed, but stored in the file.
- **White Text:** Text with font colour #FFFFFF on a white page background is invisible to the reader but fully present in the file and readable by a machine.
- **Zero-Width Characters:** Scans all XML inside the DOCX ZIP for invisible Unicode characters (U+200B Zero Width Space, U+200C Non-Joiner, etc.)
- **VBA Macros:** Checks for vbaProject.bin inside the ZIP. Macros can hide code that exfiltrates data or decodes hidden payloads.
- **Custom XML Parts:** DOCX supports arbitrary XML data stored inside the file. This is a known data hiding location.

PPTX / ODP Detection — 5 Checks

- **Hidden Slides:** Slides with show='0' attribute are not displayed during presentation but exist in the file. Full slide content is accessible.
- **Off-Canvas Shapes:** Objects positioned outside the slide boundary (negative coordinates or beyond slide width/height) are invisible during presentation but contain data.
- **White Text Boxes:** Text boxes with white font colour are invisible against white slide backgrounds.

- **Speaker Notes Steganography:** Checks speaker notes of every slide for zero-width characters and homoglyphs.
- **Zero-Width Characters:** Full XML scan of all presentation XML files.

XLSX / ODS Detection — 5 Checks

- **Hidden Sheets:** Worksheets with sheet_state = 'hidden' or 'veryHidden' are not visible in Excel but contain all their data.
- **Hidden Rows / Columns:** Rows or columns set as hidden in row_dimensions or column_dimensions — invisible in normal view.
- **White / Tiny Cells:** Cells with white font colour or font size below 4pt are effectively invisible. Checks every cell with content.
- **VBA Macros:** Same as DOCX — checks for vbaProject.bin.
- **Zero-Width Characters:** Full XML scan of all spreadsheet XML.

TXT / CSV / HTML / XML / MD Detection — 5 Checks

- **Zero-Width Characters:** Scans entire file for 15 types of invisible Unicode characters. Each one can encode binary data. Example: Space=0, Zero-Width Space=1 → binary message.
- **Trailing Whitespace:** Trailing spaces at line ends can encode bits. Flags files where more than 30% of lines have trailing spaces.
- **Homoglyphs:** Visually identical Unicode characters substituted for Latin letters. Example: Cyrillic 'a' (U+0430) looks identical to Latin 'a' but is a different character — detectable by code.
- **Line Ending Anomaly:** Files normally use consistent line endings (CRLF on Windows, LF on Unix). Mixed endings can encode bits.
- **Null Bytes:** Null characters (0x00) in text files are always suspicious — normal text never contains them.

5. Audio Steganography Detection

Audio files store sound as a sequence of numerical samples. A WAV file at 44,100 Hz stores 44,100 numbers per second per channel. Each sample is typically a 16-bit integer (-32768 to +32767). Just like images, the last bit (LSB) of each sample can be changed without any audible effect — a change of 1 in 32768 is below the threshold of human hearing.

Audio Example

Original sample: 15420 = 001111000101100

After hiding '1': 15421 = 001111000101110

Audible difference: $1/32768 = 0.003\%$ → completely inaudible

Check	How It Works
LSB Randomness	Natural audio has correlated sample values — music has smooth transitions. LSB embedding destroys this. Measures entropy of LSB stream: >0.98 with ones ratio near 0.5 is suspicious (perfectly random bits = hidden data).
Silent Segment Noise	Silent parts of audio (amplitude < 1% of peak) should have near-zero values. Hidden data embedded in silence raises the noise floor measurably. Measures RMS (Root Mean Square) energy in silent segments.
Excess Metadata	MP3 ID3 tags and WAV INFO chunks can store arbitrary data. Checks if metadata size is unusually large (>50KB or >30% of file size).
Appended Data	Data can be hidden by appending it after the audio end marker. For WAV: checks if file size exceeds RIFF header declared size. For MP3: locates last valid sync word and measures trailing bytes.

6. Video Steganography Detection

Video files are complex containers holding video frames, audio tracks, subtitles, metadata, and chapter information. This complexity provides many places to hide data. The system uses OpenCV for video analysis and raw binary parsing for container-level inspection.

Check	How It Works
Appended Data	The most common video hiding technique. Data is added after the official container end marker. For MP4: locates 'moov' atom and checks if bytes follow beyond it. For AVI: compares file size to RIFF declared size. Flags if more than 1024 bytes found after container end.
Size/Duration Anomaly	Uses OpenCV to get video duration, resolution, and FPS. Calculates expected maximum file size (200 MB/minute for HD video). If actual file size greatly exceeds this, extra data may be hidden.
Large Metadata Blocks	Scans first 64KB of file for metadata markers: 'udta' (MP4 user data), 'meta', 'XMP_' (XMP metadata), 'ID3'. Excessive metadata is suspicious.
MKV Attachments	MKV (Matroska) format supports embedded attachments — any file type can be attached. Checks for EBML Attachments element (0x1941A469). Legitimate attachments exist but are a known hiding location.

7. Graphical User Interface (GUI)

The GUI is built with Tkinter, Python's built-in GUI framework. It has a dark theme for a professional forensic tool appearance. The interface is designed to be usable by someone with no technical background.

Component	What It Shows
Left Panel	File list showing all loaded files with Risk level and Score. Four Add buttons (Images / Docs / Audio / Video). Analyse All Files button, Export PDF, Remove, Clear buttons. Current file verdict display.
Summary Tab	Risk level banner (HIGH/MEDIUM/LOW/CLEAN) with colour coding. Stats boxes: Risk Score, Tests Positive, File Size, Resolution/Type. Individual detection result cards — one per check with DETECTED/CLEAN badge. File SHA-256 hash for integrity verification.
Visual Analysis Tab	Images: LSB bit plane visualisation + pixel histograms + detection bar chart. Audio: waveform plot + LSB stream plot for first 2000 samples. Video: 6 frame thumbnails sampled evenly from the video.
Detailed Results Tab	Images: per-channel extraction table showing printable ratio, length, and the actual extracted message text if found. Documents/Audio/Video: full check breakdown with exact values found.
Raw JSON Tab	Complete machine-readable JSON output of all analysis results. Can be copied and used by other tools or stored as evidence.

Risk Level System

HIGH (score 70-100%)	Steganography LIKELY present. Strong evidence across multiple tests.
MEDIUM (score 35-69%)	Steganography POSSIBLY present. Notable indicators found.
LOW (score 10-34%)	Suspicious indicators detected. Could be false positive.
CLEAN (score 0-9%)	No steganography indicators found. File appears normal.

8. PDF Report Generation

The system generates a professional forensic PDF report using ReportLab. The report can include any number of files of any type in a single document.

Section	Contents
Cover Page	Title, date, total files analysed, HIGH/MEDIUM/LOW counts
Risk Chart	Horizontal bar chart showing risk score for every file
Per-File Section	Filename, file type, size, SHA-256 hash, thumbnail (images)
Verdict Banner	Colour-coded risk level and verdict statement
Detection Table	Every check with DETECTED/CLEAN result and key metrics
Extraction	Extracted hidden message text if found (images only)
Disclaimer	Forensic note that results are probabilistic

9. How to Use the System

Step 1 — Install Python

- Download Python 3.9+ from python.org
- During installation, tick 'Add Python to PATH'

Step 2 — Install Dependencies

```
cd unified_project
```

```
pip install -r requirements.txt
```

Step 3 — Run the Application

```
python main.py
```

Step 4 — Analyse Files

1. Add files	Click the Images / Docs / Audio / Video button and select files
2. Analyse	Click '■ Analyse All Files' — runs all checks automatically
3. View results	Click any file in the list — results appear in the four tabs
4. Export report	Click '■ Export PDF Report' to save a forensic PDF

10. Complete Detection Techniques Summary

File Type	Check Name	What It Detects
Image	LSB Analysis	Spatial correlation of last bits across pixels
Image	Chi-Square Test	Statistical equalisation of paired pixel values
Image	RS Analysis	Regular vs Singular pixel group ratio imbalance
Image	Histogram Analysis	Comb pattern in pixel value frequency distribution
Image	DCT/FFT Analysis	Elevated high-frequency coefficients
Image	Message Extraction	Reads actual LSB bits as text characters
PDF	Raw Stream Scan	JavaScript, embedded files, launch actions
PDF	Invisible Text	White colour or sub-0.5pt font size text
PDF	Blank Page Ratio	High ratio of pages with no readable content
DOCX	Hidden Text (Vanish)	w:vanish XML attribute on text runs
DOCX	White Text	Font colour #FFFFFF on white background
DOCX	Zero-Width Chars	15 types of invisible Unicode characters
DOCX	VBA Macros	Presence of vbaProject.bin in ZIP
DOCX	Custom XML Parts	Arbitrary XML data parts embedded in file
PPTX	Hidden Slides	show='0' attribute on slide elements
PPTX	Off-Canvas Shapes	Objects outside slide boundary coordinates
PPTX	White Text Boxes	White font colour in text frame runs
PPTX	Speaker Notes Stego	Zero-width chars and homoglyphs in notes
XLSX	Hidden Sheets	sheet_state = hidden or veryHidden
XLSX	Hidden Rows/Cols	Hidden row_dimensions or column_dimensions
XLSX	White/Tiny Cells	White font or font size < 4pt in cells
XLSX	VBA Macros	Presence of vbaProject.bin in ZIP
Text	Zero-Width Chars	15 invisible Unicode chars in file content
Text	Trailing Whitespace	>30% of lines have trailing spaces
Text	Homoglyphs	Cyrillic/Greek lookalike chars for Latin
Text	Line Ending Anomaly	Mixed CRLF/LF/CR line endings
Text	Null Bytes	0x00 bytes in text file
Audio	LSB Randomness	High entropy + balanced ones/zeros in LSBs
Audio	Silent Segment Noise	Elevated RMS energy in silent sections
Audio	Excess Metadata	ID3/INFO metadata > 50KB or > 30% of file
Audio	Appended Data	Bytes after RIFF size or last MP3 sync word
Video	Appended Data	Bytes after moov atom or RIFF container end
Video	Size/Duration Anomaly	File size >> expected for its duration
Video	Large Metadata	Excessive udta/meta/XMP metadata markers
Video	MKV Attachments	EBML Attachments element (0x1941A469)

11. Limitations and Future Scope

Current Limitations

- Statistical detection: results are probabilistic, not 100% certain
- Advanced stego tools (e.g. steghide with encryption) may not be detected
- Audio/Video analysis does not do frame-level pixel analysis yet
- Does not handle password-protected documents
- Video analysis relies on container metadata, not frame content

Future Scope

Deep Learning	Train a CNN model on labelled stego/clean images for higher accuracy
Steganalysis Library	Integrate aletheia or stegexpose for more advanced detection
Frame-level video	Apply image analysis to individual video frames
Network traffic	Detect steganography in HTTP images or DNS tunnelling
Database logging	Store all analysis results in SQLite for case management
Batch folder scan	Automatically scan entire folders recursively
CLI mode	Command-line interface for integration with other tools
Cloud deployment	Web interface using Flask/Django for remote access

Project Summary

This Steganography Detection System is a complete forensic analysis tool that covers 35 detection checks across 8 file categories. It combines statistical analysis, binary parsing, XML inspection, and signal processing to detect hidden data across every major file format. The graphical interface makes it accessible to non-technical users while the Raw JSON output and PDF report make it suitable for professional forensic investigations.